Just for Fun

And finally, in our short detour into the dustier corners of PHP string functions, we have… this… thing. A common exercise for computer science students (or software engineers undergoing tech tests in interviews) is to implement something called the "Caeser cipher" algorithm. This is a classic of cryptography dating from about a million years ago and is the simplest form of *substitution cipher*. We'll be learning more about those in an upcoming section. `str_rot13()` applies the ROT13 (rotate by thirteen places) transformation: each letter is moved forward in the alphabet by thirteen places, preserving case. Which means `str_rot13()` will be its own inverse for the Latin alphabet (this is an ASCII-only function).

```
print str_rot13("abcdef") . PHP_EOL;
print str_rot13("ABCdef") . PHP_EOL;
print str_rot13(str_rot13("I am not secure!"));
```

```
Output:
nopqrs
NOPqrs
I am not secure!
```

Handy if you need to make text slightly harder to read at a glance (maybe hiding spoilers on forum posts?). But that's about it. If true encryption is required, PHP has tools for that, and we'll see them soon.

Hashing and Error Checking

Adjacent to the topic of encoding is hashing. A hash is simply a string of fixed size that is calculated from another string (of arbitrary size). Unlike encoding, hash functions are not reversible, which means it is impossible to compute the original input data from the output data.

Hashes are useful for verifying the integrity of messages (a message is transmitted, a hash value is transmitted separately, and the two can be compared to ensure the message was not altered in transit), generating signatures (I sign a message with a shared secret key; other owners of the key are then able to verify both that the contents were not tampered with *and* the authenticity of the sender), and storing sensitive strings such as passwords (which should be stored as hashes – credentials are passed through an identical hashing scheme and compared to the stored hash).